

Summary

In this chapter, we covered calculating factorials and the Fibonacci sequence, two classic recursive programming problems. This chapter featured both iterative and recursive implementations for these algorithms. Despite being classic examples of recursion, their recursive algorithms suffer from critical flaws. The recursive factorial function can cause stack overflows, while the recursive Fibonacci function performs so many redundant calculations that it's far too slow to be effective in the real world.

We explored how to create recursive algorithms from iterative algorithms and how to create iterative algorithms from recursive algorithms. Iterative algorithms use a loop, and any recursive algorithm can be performed iteratively by using a loop and a stack data structure. Recursion is often an overly complicated solution, but programming problems that involve a tree-like structure and backtracking are particularly suitable for recursive implementations.

Writing recursive functions is a skill that improves with practice and experience. The rest of this book covers several well-known recursion examples and explores their strengths and limitations.